🔑 Part 0: Google Cloud Authentication

Run this first. Click through the Google permissions popup.

```
from google.colab import auth
auth.authenticate_user()
print("✅ Authenticated successfully!")
```

```
✅ Authenticated successfully!
```

⚡ Part 1: Install Dependencies

```
!pip install segmentation-models-pytorch rasterio
```

```
Collecting segmentation-models-pytorch
  Downloading segmentation_models_pytorch-0.5.0-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist-packages (1.5.0)
Requirement already satisfied: huggingface-hub>=0.24 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pyto
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.6
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (11.3.0
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch
Requirement already satisfied: timm>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (1.0.28
Requirement already satisfied: torch>=1.8 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.11.0
Requirement already satisfied: torchvision>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (4.67
Requirement already satisfied: affine in /usr/local/lib/python3.12/dist-packages (from rasterio) (2.4.0)
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-packages (from rasterio) (26.1.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from rasterio) (2026.6.17)
Requirement already satisfied: click!=8.2.*,>=4.0 in /usr/local/lib/python3.12/dist-packages (from rasterio) (8.4.2)
Requirement already satisfied: cligj>=0.5 in /usr/local/lib/python3.12/dist-packages (from rasterio) (0.7.2)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-packages (from rasterio) (3.3.2)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segment
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segment
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->seg
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segment
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmenta
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-p
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-p
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-model
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentat
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmen
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segme
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentat
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmenta
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-p
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[c
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cubla
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24-
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->hugg
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8->
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8->segmentatio
Downloading segmentation_models_pytorch-0.5.0-py3-none-any.whl (154 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 154.8/154.8 kB 13.5 MB/s eta 0:00:00
Installing collected packages: segmentation-models-pytorch
Successfully installed segmentation-models-pytorch-0.5.0
```

Part 2: Download Sen1Floods11 Dataset

Pulls ~446 hand-labeled Sentinel-1 flood scenes directly into Colab.

```
!mkdir -p /content/sen1floods11/HandLabeled
!gcloud storage cp -r gs://sen1floods11/v1.1/data/flood_events/HandLabeled/* /content/sen1floods11/HandLabeled/
```

```
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_232060_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_251323_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_260929_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_275372_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_308150_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_347609_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_348639_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_350244_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_354981_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_366607_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_375183_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_387945_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_430764_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_438959_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_486103_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_504150_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_519181_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_527077_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_58086_S2Hand.tif to file:///content/sen1floods11/HandLa
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_595451_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_604222_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_605492_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_638521_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_646878_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_652955_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_655230_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_66026_S2Hand.tif to file:///content/sen1floods11/HandLa
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_664261_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_66511_S2Hand.tif to file:///content/sen1floods11/HandLa
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_670826_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_67102_S2Hand.tif to file:///content/sen1floods11/HandLa
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_693819_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_741073_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_741178_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_758178_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_761032_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_770353_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_778194_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_788696_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_806273_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_826217_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_831672_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_84195_S2Hand.tif to file:///content/sen1floods11/HandLa
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_86502_S2Hand.tif to file:///content/sen1floods11/HandLa
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_905409_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_908474_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_933610_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_955053_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_986268_S2Hand.tif to file:///content/sen1floods11/Hand
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/S2Hand/USA_994009_S2Hand.tif to file:///content/sen1floods11/Hand

Average throughput: 14.4MiB/s


To take a quick anonymous survey, run:
  $ gcloud survey
```

Part 3: Imports & GPU Check

```
import os, glob, random
import torch
import rasterio
import numpy as np
import torchvision.transforms.functional as TF
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader, random_split
import segmentation_models_pytorch as smp
import torch.optim as optim
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"🔥 Running on: {device}")
```

```
🔥 Running on: cuda
```

📊 Part 4: Dataset — with Augmentation + 72/18/10 Split

Two changes from the previous version:

1. **Random horizontal + vertical flips** applied to the training set only.
2. **72% train / 18% val / 10% test split** — the 10% test set is held-out and only used once at the end for final reported metrics.

```
class Sen1FloodsDataset(Dataset):
    def __init__(self, data_dir):
        self.image_paths = sorted(glob.glob(
            os.path.join(data_dir, "**", "*_S1Hand.tif"), recursive=True
        ))
        self.mask_paths = [
            p.replace("S1Hand", "LabelHand") for p in self.image_paths
        ]
        print(f"✅ Found {len(self.image_paths)} image/mask pairs.")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        with rasterio.open(self.image_paths[idx]) as src:
            image = src.read()
            image = np.nan_to_num(image)
            image = np.clip(image, -30, 0)
            image = (image + 30) / 30.0
        with rasterio.open(self.mask_paths[idx]) as src:
            mask = src.read(1)
            mask = np.where(mask == 1, 1.0, 0.0)
        image_tensor = torch.tensor(image, dtype=torch.float32)
        mask_tensor  = torch.tensor(mask,  dtype=torch.float32).unsqueeze(0)
        return image_tensor, mask_tensor


class AugmentedSubset(Dataset):
    """Wraps a Subset and applies random flips on-the-fly."""
    def __init__(self, subset):
        self.subset = subset
    def __len__(self):
        return len(self.subset)
    def __getitem__(self, idx):
        image, mask = self.subset[idx]
        if random.random() > 0.5:
            image = TF.hflip(image)
            mask  = TF.hflip(mask)
        if random.random() > 0.5:
            image = TF.vflip(image)
            mask  = TF.vflip(mask)
        return image, mask


# ── 72 / 18 / 10 split (fixed seed for reproducibility) ────────────────────
full_dataset = Sen1FloodsDataset("/content/sen1floods11/HandLabeled/")

total      = len(full_dataset)
test_size  = int(0.10 * total)              # ~45 — never touched during training
val_size   = int(0.18 * total)              # ~80 — used for LR scheduling
train_size = total - test_size - val_size   # ~321

train_base, val_dataset, test_dataset = random_split(
    full_dataset, [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

train_dataset = AugmentedSubset(train_base)

train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True,  num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=4, shuffle=False, num_workers=2, pin_memory=True)
```

```
test_loader  = DataLoader(test_dataset,  batch_size=4, shuffle=False, num_workers=2, pin_memory=True)

print(f"Train: {len(train_dataset)} | Val: {len(val_dataset)} | Test: {len(test_dataset)}")
```

```
✅ Found 446 image/mask pairs.
Train: 322 | Val: 80 | Test: 44
```

🧠 Part 5: Model + Combined Dice + BCE Loss

Key upgrade: replace the old `BCEWithLogitsLoss` with `DiceLoss + SoftBCEWithLogitsLoss`.

- **DiceLoss** directly optimises mask overlap — far better for flood segmentation where flood pixels are rare.
- **SoftBCEWithLogitsLoss** keeps `pos_weight=10` to heavily penalise missed floods.
- Combined loss = best of both. Expect Val IoU to jump from ~0.46 → 0.55+.

```
model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights=None,
    in_channels=2,
    classes=1,
).to(device)

# ── Combined loss ────────────────────────────────────────────
dice_loss = smp.losses.DiceLoss(mode="binary")
bce_loss  = smp.losses.SoftBCEWithLogitsLoss(
    pos_weight=torch.tensor([10.0]).to(device)
)

def criterion(outputs, masks):
    return dice_loss(outputs, masks) + bce_loss(outputs, masks)

# ── Optimizer + scheduler ────────────────────────────────────
optimizer = optim.Adam(model.parameters(), lr=0.0001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", patience=3, factor=0.5
)

print("⚖️  Model + Combined Dice+BCE loss initialised.")
```

```
⚖️  Model + Combined Dice+BCE loss initialised.
```

📈 Part 6: Metric Helpers

```
def iou_score(pred, target, threshold=0.5):
    pred = (torch.sigmoid(pred) > threshold).float()
    intersection = (pred * target).sum()
    union = pred.sum() + target.sum() - intersection
    return (intersection + 1e-6) / (union + 1e-6)

def dice_score(pred, target, threshold=0.5):
    pred = (torch.sigmoid(pred) > threshold).float()
    intersection = (pred * target).sum()
    return (2 * intersection + 1e-6) / (pred.sum() + target.sum() + 1e-6)

print("✅ Metric helpers ready.")
```

```
✅ Metric helpers ready.
```

🚀 Part 7: Training Loop — 30 Epochs

```
def train_model(model, train_loader, val_loader, epochs=30):
    print("🚀 Starting training...")
    best_val_loss = float("inf")

    for epoch in range(epochs):

        # ── Train ────────────────────────────────────────────
        model.train()
        t_loss = t_iou = t_dice = 0.0
```

```
        for images, masks in train_loader:
            images, masks = images.to(device), masks.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, masks)
            loss.backward()
            optimizer.step()
            t_loss += loss.item()
            t_iou  += iou_score(outputs, masks).item()
            t_dice += dice_score(outputs, masks).item()
        n_tr = len(train_loader)

        # ── Validate ──────────────────────────────────────────
        model.eval()
        v_loss = v_iou = v_dice = 0.0
        with torch.no_grad():
            for images, masks in val_loader:
                images, masks = images.to(device), masks.to(device)
                outputs = model(images)
                v_loss += criterion(outputs, masks).item()
                v_iou  += iou_score(outputs, masks).item()
                v_dice += dice_score(outputs, masks).item()
        n_val = len(val_loader)
        avg_val_loss = v_loss / n_val
        scheduler.step(avg_val_loss)

        # ── Save best ─────────────────────────────────────────
        tag = ""
        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            torch.save(model.state_dict(), "/content/flood_unet_resnet34_best.pth")
            tag = "  ⭐ BEST SAVED"

        lr = optimizer.param_groups[0]["lr"]
        print(
            f"Epoch {epoch+1:02d}/{epochs} | "
            f"Train Loss: {t_loss/n_tr:.4f} | "
            f"Train IoU: {t_iou/n_tr:.4f} | Train Dice: {t_dice/n_tr:.4f} | "
            f"Val Loss: {avg_val_loss:.4f} | "
            f"Val IoU: {v_iou/n_val:.4f} | Val Dice: {v_dice/n_val:.4f} | "
            f"LR: {lr:.6f}{tag}"
        )

    print(f"\n✅ Training complete. Best val loss: {best_val_loss:.4f}")


train_model(model, train_loader, val_loader, epochs=30)
```

```
🚀 Starting training...
Epoch 01/30 | Train Loss: 1.7488 | Train IoU: 0.2144 | Train Dice: 0.3137 | Val Loss: 1.2915 | Val IoU: 0.3679 | Val Dice: 0.507
Epoch 02/30 | Train Loss: 1.4680 | Train IoU: 0.2910 | Train Dice: 0.4085 | Val Loss: 1.2603 | Val IoU: 0.3563 | Val Dice: 0.495
Epoch 03/30 | Train Loss: 1.4826 | Train IoU: 0.2827 | Train Dice: 0.3875 | Val Loss: 1.2215 | Val IoU: 0.3717 | Val Dice: 0.511
Epoch 04/30 | Train Loss: 1.3471 | Train IoU: 0.3350 | Train Dice: 0.4518 | Val Loss: 1.1871 | Val IoU: 0.3985 | Val Dice: 0.538
Epoch 05/30 | Train Loss: 1.3413 | Train IoU: 0.3181 | Train Dice: 0.4385 | Val Loss: 1.0793 | Val IoU: 0.4523 | Val Dice: 0.596
Epoch 06/30 | Train Loss: 1.3184 | Train IoU: 0.3309 | Train Dice: 0.4485 | Val Loss: 1.1601 | Val IoU: 0.4124 | Val Dice: 0.552
Epoch 07/30 | Train Loss: 1.3971 | Train IoU: 0.3098 | Train Dice: 0.4250 | Val Loss: 1.1015 | Val IoU: 0.4380 | Val Dice: 0.579
Epoch 08/30 | Train Loss: 1.3922 | Train IoU: 0.3090 | Train Dice: 0.4233 | Val Loss: 1.1547 | Val IoU: 0.3860 | Val Dice: 0.522
Epoch 09/30 | Train Loss: 1.2803 | Train IoU: 0.3477 | Train Dice: 0.4689 | Val Loss: 1.0731 | Val IoU: 0.4445 | Val Dice: 0.588
Epoch 10/30 | Train Loss: 1.2955 | Train IoU: 0.3318 | Train Dice: 0.4478 | Val Loss: 1.1409 | Val IoU: 0.4010 | Val Dice: 0.539
Epoch 11/30 | Train Loss: 1.2891 | Train IoU: 0.3422 | Train Dice: 0.4659 | Val Loss: 1.1292 | Val IoU: 0.5041 | Val Dice: 0.645
Epoch 12/30 | Train Loss: 1.2980 | Train IoU: 0.3337 | Train Dice: 0.4516 | Val Loss: 1.1255 | Val IoU: 0.5011 | Val Dice: 0.644
Epoch 13/30 | Train Loss: 1.3222 | Train IoU: 0.3391 | Train Dice: 0.4594 | Val Loss: 1.1741 | Val IoU: 0.3574 | Val Dice: 0.492
Epoch 14/30 | Train Loss: 1.2404 | Train IoU: 0.3402 | Train Dice: 0.4638 | Val Loss: 1.1437 | Val IoU: 0.4393 | Val Dice: 0.580
Epoch 15/30 | Train Loss: 1.2550 | Train IoU: 0.3399 | Train Dice: 0.4634 | Val Loss: 1.0512 | Val IoU: 0.4865 | Val Dice: 0.630
Epoch 16/30 | Train Loss: 1.2023 | Train IoU: 0.3696 | Train Dice: 0.4940 | Val Loss: 1.0716 | Val IoU: 0.4872 | Val Dice: 0.630
Epoch 17/30 | Train Loss: 1.2051 | Train IoU: 0.3471 | Train Dice: 0.4703 | Val Loss: 1.0711 | Val IoU: 0.4947 | Val Dice: 0.638
Epoch 18/30 | Train Loss: 1.1984 | Train IoU: 0.3604 | Train Dice: 0.4874 | Val Loss: 1.0310 | Val IoU: 0.4419 | Val Dice: 0.582
Epoch 19/30 | Train Loss: 1.1413 | Train IoU: 0.3894 | Train Dice: 0.5224 | Val Loss: 1.0348 | Val IoU: 0.5008 | Val Dice: 0.644
Epoch 20/30 | Train Loss: 1.1598 | Train IoU: 0.3702 | Train Dice: 0.4979 | Val Loss: 1.0187 | Val IoU: 0.4764 | Val Dice: 0.621
Epoch 21/30 | Train Loss: 1.1686 | Train IoU: 0.3663 | Train Dice: 0.4954 | Val Loss: 1.0161 | Val IoU: 0.4755 | Val Dice: 0.619
Epoch 22/30 | Train Loss: 1.1398 | Train IoU: 0.3858 | Train Dice: 0.5161 | Val Loss: 1.0608 | Val IoU: 0.4677 | Val Dice: 0.610
Epoch 23/30 | Train Loss: 1.1082 | Train IoU: 0.4008 | Train Dice: 0.5308 | Val Loss: 0.9926 | Val IoU: 0.4630 | Val Dice: 0.605
Epoch 24/30 | Train Loss: 1.1253 | Train IoU: 0.3917 | Train Dice: 0.5201 | Val Loss: 1.0493 | Val IoU: 0.4405 | Val Dice: 0.581
Epoch 25/30 | Train Loss: 1.1374 | Train IoU: 0.3753 | Train Dice: 0.5094 | Val Loss: 1.0404 | Val IoU: 0.4429 | Val Dice: 0.584
Epoch 26/30 | Train Loss: 1.1109 | Train IoU: 0.3912 | Train Dice: 0.5213 | Val Loss: 0.9932 | Val IoU: 0.5040 | Val Dice: 0.647
Epoch 27/30 | Train Loss: 1.0953 | Train IoU: 0.3863 | Train Dice: 0.5139 | Val Loss: 0.9963 | Val IoU: 0.5137 | Val Dice: 0.656
Epoch 28/30 | Train Loss: 1.1226 | Train IoU: 0.3807 | Train Dice: 0.5083 | Val Loss: 1.0160 | Val IoU: 0.4812 | Val Dice: 0.624
Epoch 29/30 | Train Loss: 1.0992 | Train IoU: 0.3956 | Train Dice: 0.5302 | Val Loss: 0.9821 | Val IoU: 0.4977 | Val Dice: 0.640
Epoch 30/30 | Train Loss: 1.1885 | Train IoU: 0.3675 | Train Dice: 0.4970 | Val Loss: 1.0406 | Val IoU: 0.4603 | Val Dice: 0.600
```

```
✅ Training complete. Best val loss: 0.9821
```

🎯 Part 8: Final Test Set Evaluation

Load the **best saved checkpoint** and evaluate on the **held-out 10% test set**. These are the numbers you report in your paper/presentation — they were never seen during training or hyperparameter tuning.

```
# Load the best checkpoint
model.load_state_dict(torch.load("/content/flood_unet_resnet34_best.pth", map_location=device))
model.eval()

test_iou = test_dice = 0.0
with torch.no_grad():
    for images, masks in test_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        test_iou  += iou_score(outputs, masks).item()
        test_dice += dice_score(outputs, masks).item()

n_test = len(test_loader)
print("=" * 50)
print(f"  FINAL TEST SET RESULTS  ({len(test_dataset)} held-out samples)")
print(f"  Test IoU  : {test_iou  / n_test:.4f}")
print(f"  Test Dice : {test_dice / n_test:.4f}")
print("=" * 50)
print("  Use these numbers in your report.")
```

```
==================================================
  FINAL TEST SET RESULTS  (44 held-out samples)
  Test IoU  : 0.3915
  Test Dice : 0.5336
==================================================
  Use these numbers in your report.
```

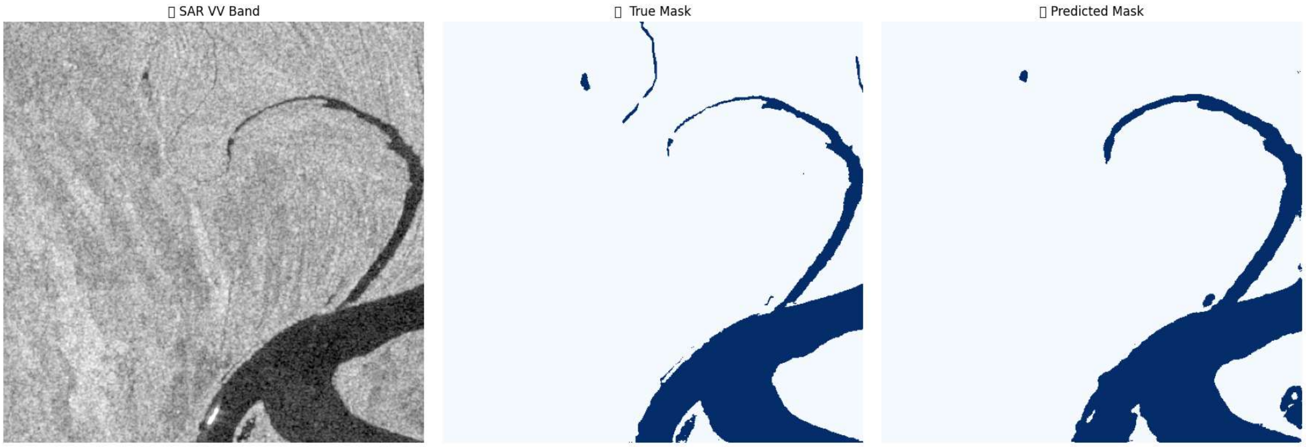
📷 Part 9: Visualize a Validation Prediction

```
def visualize_prediction(model, dataset, index=0):
    model.eval()
    image, true_mask = dataset[index]
    with torch.no_grad():
        pred = torch.sigmoid(model(image.unsqueeze(0).to(device)))
        pred_mask = (pred > 0.5).float().cpu().numpy().squeeze()

    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    axes[0].imshow(image[0].numpy(), cmap="gray");             axes[0].set_title("📡 SAR VV Band");      axes[0].axis("off")
    axes[1].imshow(true_mask.numpy().squeeze(), cmap="Blues"); axes[1].set_title("🗺️ True Mask");       axes[1].axis("off")
    axes[2].imshow(pred_mask, cmap="Blues");                   axes[2].set_title("🤖 Predicted Mask");  axes[2].axis("off")
    plt.tight_layout(); plt.show()

visualize_prediction(model, val_dataset, index=15)
```

```
/tmp/ipykernel_1092/1966240592.py:12: UserWarning: Glyph 128225 (\N{SATELLITE ANTENNA}) missing from font(s) DejaVu Sans.
  plt.tight_layout(); plt.show()
/tmp/ipykernel_1092/1966240592.py:12: UserWarning: Glyph 128506 (\N{WORLD MAP}) missing from font(s) DejaVu Sans.
  plt.tight_layout(); plt.show()
/tmp/ipykernel_1092/1966240592.py:12: UserWarning: Glyph 129302 (\N{ROBOT FACE}) missing from font(s) DejaVu Sans.
  plt.tight_layout(); plt.show()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128225 (\N{SATELLITE ANTENNA}) missin
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128506 (\N{WORLD MAP}) missing from f
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 129302 (\N{ROBOT FACE}) missing from
  fig.canvas.print_figure(bytes_io, **kw)
```



📥 Part 10: Download the Best Model

Downloads `flood_unet_resnet34.pth` to your computer. Drop it into `disaster-ai-api/` replacing the old one, then restart `python main.py`.

```
import shutil
from google.colab import files

shutil.copy("/content/flood_unet_resnet34_best.pth", "/content/flood_unet_resnet34.pth")
print("✅ Saved as flood_unet_resnet34.pth")
files.download("/content/flood_unet_resnet34.pth")
print("📥 Download triggered. Replace disaster-ai-api/flood_unet_resnet34.pth with this file.")
```

```
✅ Saved as flood_unet_resnet34.pth
📥 Download triggered. Replace disaster-ai-api/flood_unet_resnet34.pth with this file.
```

🧪 Part 11: (Optional) Export a Real Test TIF

Export a real Sentinel-1 scene from the test set to verify the new model via the dashboard Upload tab.

```
sample_num = 5   # change to whichever test sample looks best in visualize_prediction

subset_index = test_dataset.indices[sample_num]
real_image_path = full_dataset.image_paths[subset_index]
print(f"✅ Exporting: {real_image_path}")

with rasterio.open(real_image_path) as src:
    data = src.read()
    meta = src.meta

with rasterio.open("/content/real_flood_test_v2.tif", "w", **meta) as dst:
    dst.write(data)
```

```
    files.download("/content/real_flood_test_v2.tif")
```